

Sistema de controle de movimentações de conta

Documento de arquitetura e design técnico — API .NET + React

1. Contexto do desafio

O desafio pede uma API em C# e .NET para controlar movimentações financeiras de uma conta empresarial: registrar entradas e saídas, consultar saldo e consultar o histórico, garantindo consistência e nunca permitindo saldo negativo. Não há arquitetura ou banco de dados obrigatórios, e o enunciado pede explicitamente para evitar complexidade desnecessária. Como diferencial, o desafio sugere uma interface em React consumindo a API.

A solução aqui descrita cobre o mínimo pedido e adiciona três extensões justificadas: criação de conta com autenticação simples (CPF + senha), auditoria das operações e idempotência para evitar duplicidade de movimentações.

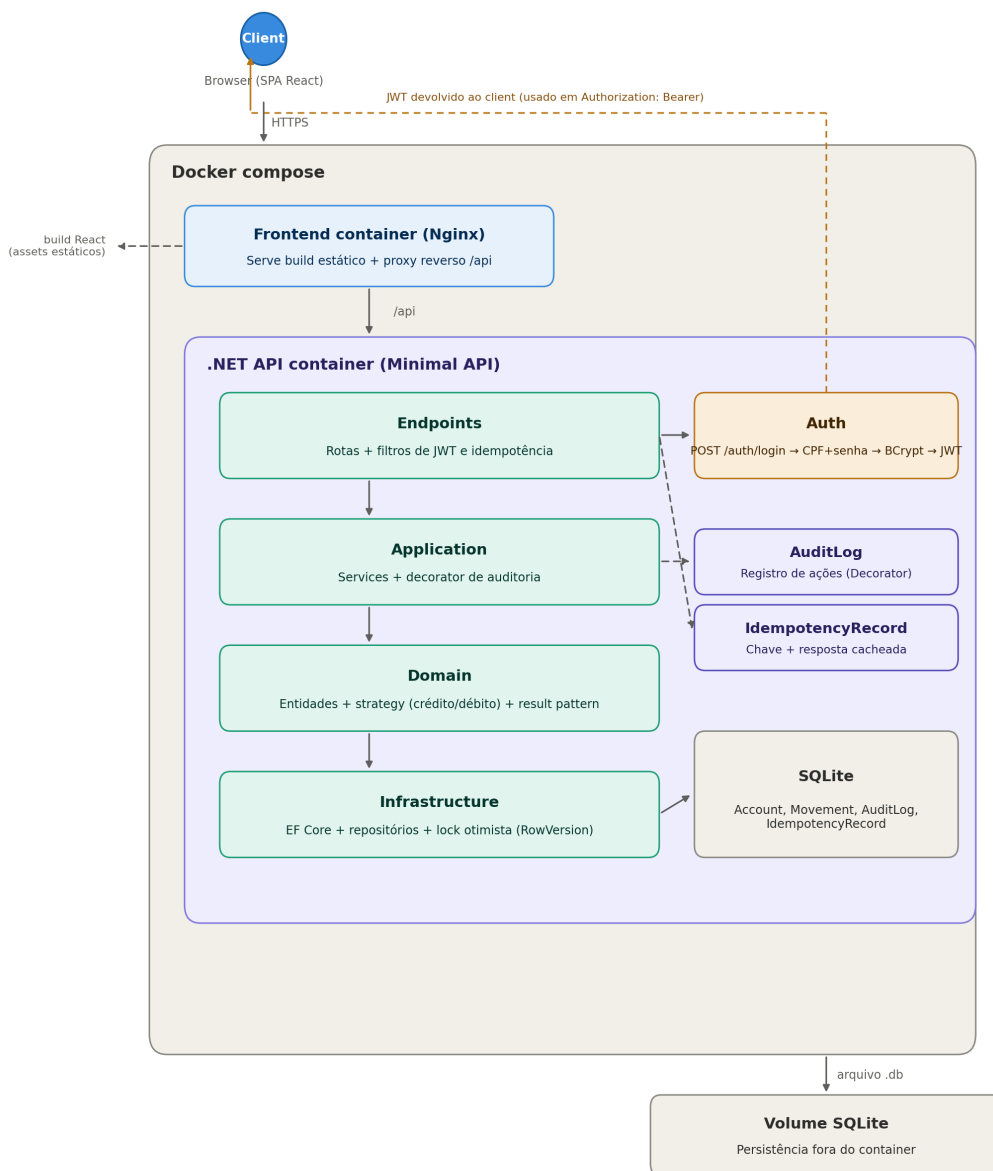
2. Arquitetura geral

A API segue Minimal API com uma Clean Architecture enxuta, sem os rótulos e camadas extras de uma arquitetura hexagonal completa — a decisão foi deliberada, para não inflar um domínio simples com abstrações que o desafio não pede.

Camada	Responsabilidade
Api	Endpoints Minimal API, filtros (JWT, idempotência), middlewares, DI
Application	Services, DTOs, interfaces, decorator de auditoria
Domain	Entidades (Account, Movement), regras de negócio, strategies, exceptions
Infrastructure	EF Core, DbContext, repositórios, seed de dados

Fluxo de dependência: Api → Application → Domain, com Infrastructure implementando as interfaces definidas pelo Domain/Application (inversão de dependência via DI nativa do .NET).

Diagrama do system design



3. Design patterns aplicados

- **Strategy** — cada tipo de movimentação (crédito/débito) valida e aplica sua própria regra sobre o saldo, evitando condicionais espalhadas pelo código.
- **Decorator** — a auditoria envolve os services (AuditedAccountService, AuditedMovementService) sem misturar log de auditoria com regra de negócio.
- **Idempotency Filter** — um IEndpointFilter intercepta requisições com o header Idempotency-Key, evitando duplicidade de operações críticas.
- **Repository** — interfaces específicas (IAccountRepository, IMovementRepository), sem repositório genérico, só o necessário para testar.
- **Result pattern** — erros de regra de negócio (ex: saldo insuficiente) retornam como valor, não como exception de fluxo de controle.

4. Modelo de domínio

Account

Id, AccountNumber (gerado automaticamente), Name, CPF (único), AccountType (Corrente/Poupança, apenas cosmético — sem regra de negócio diferente entre os tipos), PasswordHash, Balance, CreatedAt.

Movement

Id, AccountId, Type (Credit/Debit), Amount, Timestamp, resultado aplicado via strategy.

AuditLog

Id, EntityType, EntityId, Action, Payload (JSON), Timestamp.

IdempotencyRecord

Key (PK), RequestPath, RequestHash, ResponseStatusCode, ResponseBody, CreatedAt.

5. Consistência e concorrência

O sistema precisa proteger dois cenários de concorrência distintos na mesma conta:

- **Débitos concorrentes** — o caso crítico. Duas saídas de R\$80 numa conta com saldo R\$100 podem, numa implementação ingênua ("verifica saldo → debita"), passar ambas pela validação antes de qualquer uma escrever, resultando em saldo negativo.
- **Créditos concorrentes** — não ameaçam a regra de saldo negativo, mas sofrem do clássico *lost update*: duas threads leem saldo 100 e cada uma soma 50 sem saber da outra, resultando em 150 em vez de 200.

A solução adota **lock otimista** via RowVersion no EF Core: qualquer segunda escrita concorrente na mesma linha da conta — seja crédito ou débito — detecta que a versão mudou, falha com `DbUpdateConcurrencyException` e o service faz retry recarregando o saldo atualizado. Isso serializa efetivamente as escritas por conta, sem exigir lock pessimista no banco, e cobre os dois cenários com a mesma proteção.

6. Autenticação

Autenticação simples por CPF + senha, sem exigir um sistema de auth robusto — a própria Account é a identidade, sem tabela de usuário separada.

- `POST /auth/login` recebe CPF e senha, devolve um JWT.
- JWT simples, sem refresh token, sem roles.
- Middleware de autorização garante que a conta só acessa os próprios dados (`accountId` do token confere com o da rota).
- Senhas com hash via BCrypt.

7. Endpoints

Método	Rota	Observação
POST	<code>/accounts</code>	Cria conta (Idempotency-Key opcional)
POST	<code>/auth/login</code>	Autentica por CPF + senha, devolve JWT
POST	<code>/accounts/{id}/movements</code>	Registra entrada/saída (Idempotency-Key obrigatório)
GET	<code>/accounts/{id}/balance</code>	Consulta saldo disponível
GET	<code>/accounts/{id}/movements</code>	Histórico de movimentações, paginado

8. Persistência

EF Core + SQLite, com seed de 2-3 contas de teste (CPF e senha documentados no README) via `HasData()` ou `DbInitializer` no startup, para quem for avaliar conseguir logar sem precisar criar conta manualmente.

9. Docker

Docker Compose com três peças: container da API (Dockerfile multi-stage, build SDK → runtime ASP.NET), container do frontend (build Node → Nginx servindo o build estático, com proxy reverso para /api) e um volume para persistir o arquivo SQLite entre reinícios.

Comando único para subir tudo:

```
docker compose up --build
```

Frontend acessível em <http://localhost> (ou porta configurada), API em <http://localhost/api> via proxy do Nginx.

10. Testes

- **Testes unitários** no domínio — foco na regra de saldo negativo e nas strategies de crédito/débito.
- **Testes de integração** na API usando WebApplicationFactory + SQLite in-memory, cobrindo os fluxos completos de criação de conta, login, movimentação e consulta.

11. Decisões de escopo

Para manter o desafio fiel ao pedido de evitar complexidade desnecessária, ficaram fora do escopo: hierarquia de permissões (conta pai/filha), refresh token, múltiplos papéis de usuário, recuperação de senha (o frontend apenas indica humoradamente que o usuário deve "se dirigir à agência") e endpoints de edição/exclusão de conta. Essas escolhas, e as razões por trás delas, estão descritas no README como possíveis evoluções futuras.